

FAST

FORMAÇÃO ACELERADA EM
SOLUÇÕES DE TECHDESIGN



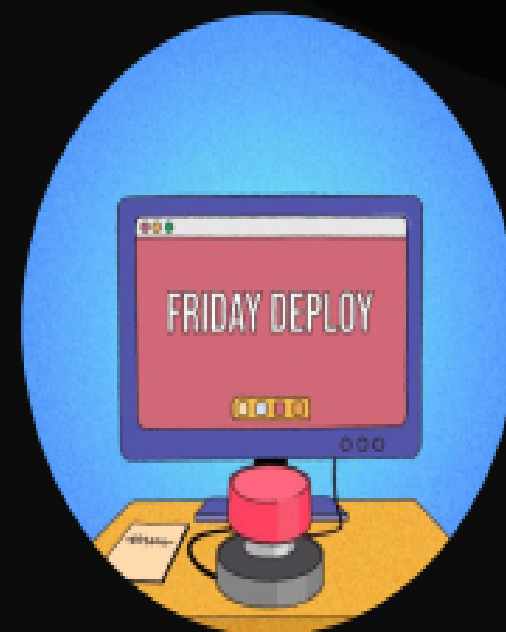
C.E.S.A.A.

QA em Ambiente DEVOPS

Módulo 6 – Aula 1

Todos os direitos reservados.

Este material, ou qualquer parte dele, não pode ser reproduzido,
divulgado ou usado de forma alguma sem autorização escrita.





Olá! Eu sou Natasha Barbachan

Engenheira de Qualidade de Software no CESAR há 6 anos, somando 9 anos de experiência na área, graduada em Análise e Desenvolvimento de Sistemas pelo IFPE. Certificada pela ISTQB com a CTFL (Certified Tester Foundation Level). Aluna do FAST em 2024 no módulo de Engenharia de Plataforma.

Linkedin:

<https://www.linkedin.com/in/natashabarbachan/>

Email Institucional:

nab@cesar.org.br

O que veremos hoje?

- Conteúdo 01 – Boas-vindas e Introdução ao Módulo
- Conteúdo 02 – Cultura DevOps e Papel do QA
- Conteúdo 03 – Integração e Ferramentas
- Conteúdo 04 – Qualidade e Feedback Contínuo



Quando vocês ouvem a palavra DevOps, o que vem à mente?

Acesse menti.com e use o código 1899 1595

QA em Ambiente DevOps

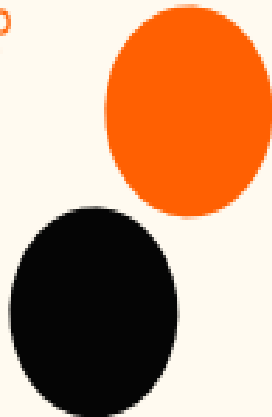


Objetivo da Trilha:

Demonstrar como o QA se integra à cultura DevOps, colaborando com a automação e qualidade contínua em pipelines CI/CD.

Questões para Reflexão:

- Como o papel do QA evoluiu com DevOps?
- Qual é a colaboração do QA?
- Quais conhecimentos o QA precisa para atuar em um ambiente DevOps?
- Como essas mudanças impactam o dia a dia do QA?



Nova Certificação Quality in DevOps (CT-QDO) pelo BSTQB

Link: <https://bstqb.qa/ct-qdo>

Material de Estudo: **Em Breve**

Ficha do Exame

Pré-requisitos: possuir a certificação CTFL

Idioma: Língua Portuguesa (Brasil)

Número de questões: 40

Tipo de questões: múltipla escolha

Tempo de Exame: 60 min (estrangeiros: +25%)

São acrescidos 15min para preenchimento dos dados do candidato e as respostas no gabarito.

Pontuação: 48 pontos (1 a 3 pontos por questão)

Aprovação: mínimo de 65% de acertos ou 31 pontos

Distribuição das questões e pontuações:

Capítulo	Questões	Pontuação
1	8	
2	9	
3	13	
4	10	

 Imagem retirada do site oficial do BSTQB (28/07/2025)



2. Cultura DevOps e Papel do QA

Reconhecer o papel estratégico do QA dentro da cultura DevOps.

Aula 1



1M+ VIEWS

What is
DevOps?

<https://youtu.be/XrgkO23l4II>

O que é DevOps?

De acordo com o **BSTQB**, DevOps é uma abordagem organizacional que visa a criar sinergia, fazendo com que o **desenvolvimento** (incluindo os testes) e as **operações trabalhem juntos** para atingir um conjunto de objetivos comuns. O DevOps exige uma mudança cultural em uma organização para preencher as lacunas entre o desenvolvimento (incluindo testes) e as operações, tratando suas funções com o mesmo valor.

O que o DevOps promove?

- A autonomia da Equipe;
- Feedback Rápido;
- Ferramentas Integradas e práticas técnicas como Integração Contínua e Entrega Contínua.

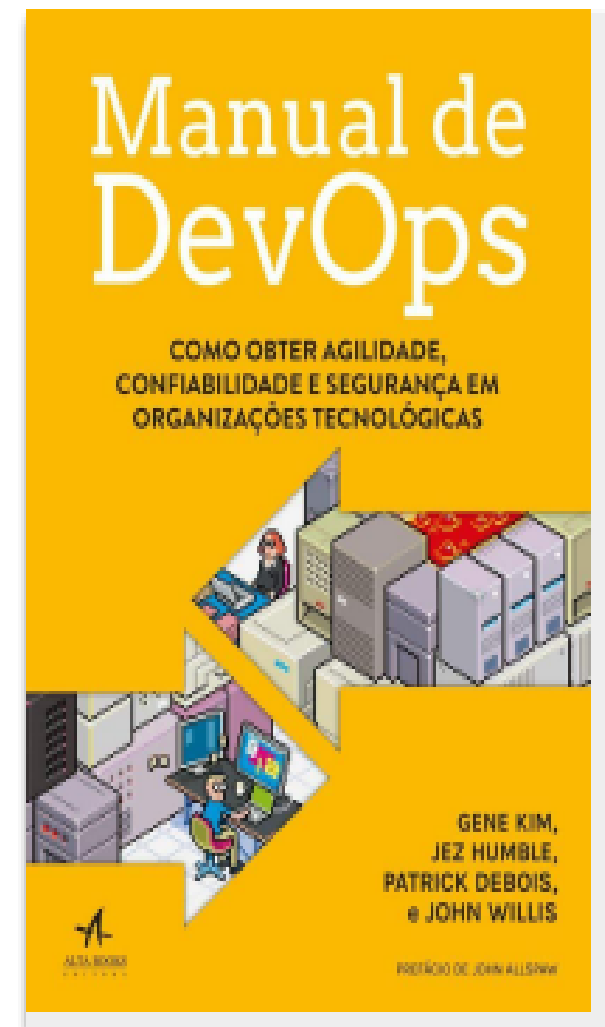
Benefícios do DevOps

- Feedback rápido sobre a qualidade do código e se as alterações afetam negativamente o código existente;
- O CI promove uma abordagem shift-left nos testes, incentivando os desenvolvedores a enviar códigos de alta qualidade acompanhados de testes de componentes e análise estática;
- Promove processos automatizados, como CI/CD, que facilitam o estabelecimento de ambientes de teste estáveis;
- Aumenta a visão das características de qualidade não funcionais (p. ex., performance, confiabilidade);
- A automação por meio de um pipeline de entrega reduz a necessidade de testes manuais repetitivos;
- O risco na regressão é minimizado devido à escala e ao alcance dos testes de regressão automatizados.

✨ **Dica de Leitura Técnica**

"Isso permite que as equipes criem, testem e liberem códigos de alta qualidade mais rapidamente por meio de um pipeline de entrega de DevOps"

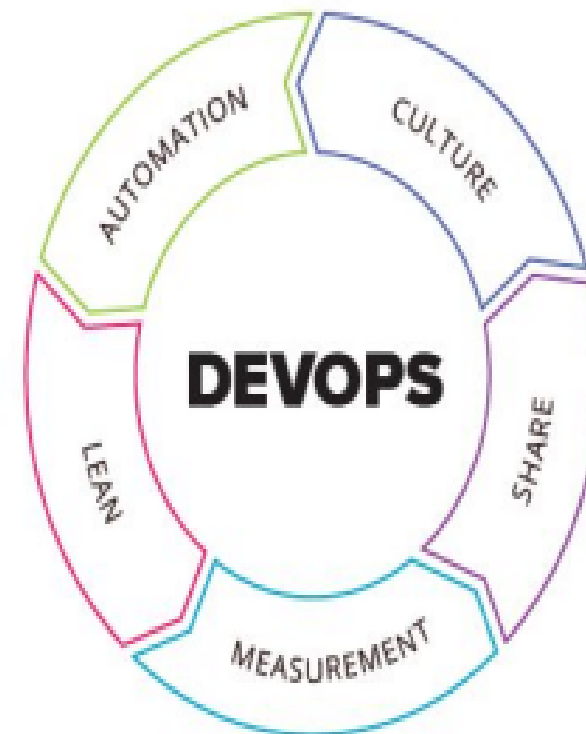
([Manual de DevOps](#), Kim et al., 2016)



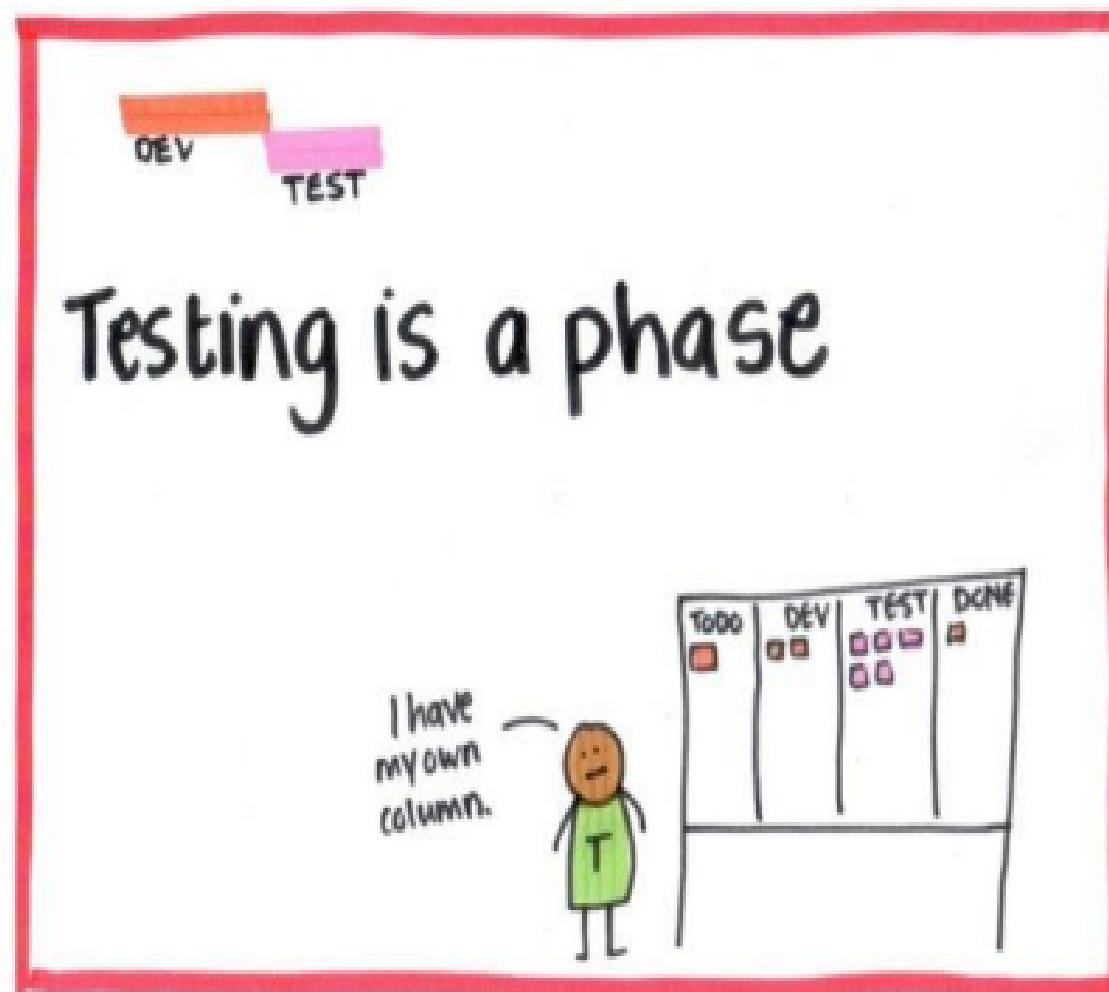
Práticas Adotadas no DEVOPS

O termo **CALMS** foi introduzido por Jez Humble, um dos autores do livro "[The DevOps Handbook](#)" mencionado anteriormente. Esse modelo é composto por cinco pilares que representam os princípios essenciais da Cultura DevOps:

- Cultura (**C**ulture)
- Automação (**A**utomation)
- Fluxo/Enxuto (**L**ean)
- Medição (**M**easure)
- Compartilhamento (**S**haring)



Como o Manifesto do Teste Ágil colaborou para a cultura devops e para a transformação do papel de QA ao longo do tempo?



Como o Manifesto do Teste Ágil colaborou para a cultura devops e para a transformação do papel de QA ao longo do tempo?



Papel Tradicional do QA vs Papel no DevOps

Aspecto	QA Tradicional	QA em DevOps
Fase de atuação	Final do ciclo (pós-desenvolvimento)	Início ao fim (ciclo completo – shift-left)
Responsabilidade	Identificar e reportar bugs	Garantir qualidade contínua e colaborativa
Interação com o time	Isolada	Colaboração constante com Dev, Ops e Produto
Foco	Validação manual, testes funcionais	Automação, performance, segurança, testes em CI/CD

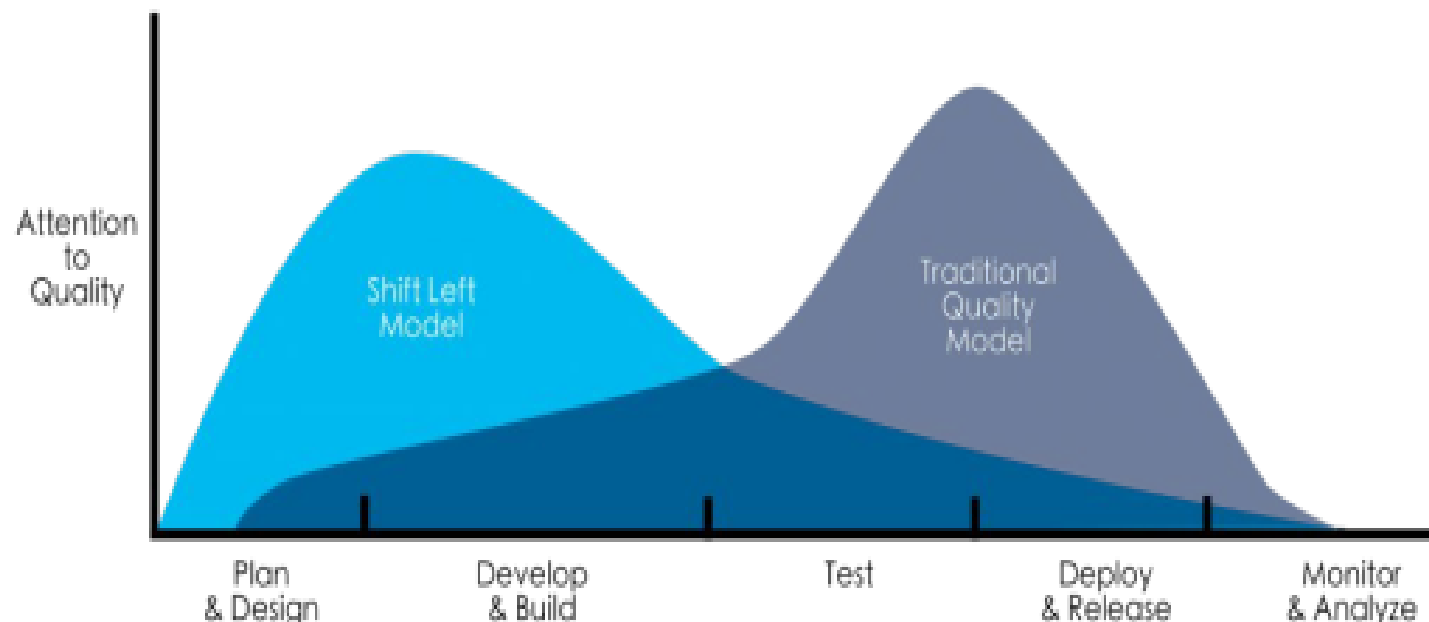
Papel Tradicional do QA vs Papel no DevOps

Aspecto	QA Tradicional	QA em DevOps
Ferramentas	Ferramentas manuais e scripts isolados	Integração com pipelines de CI/CD e monitoramento
Velocidade de entrega	Pode causar gargalos	Contribui para entregas rápidas e confiáveis
Métrica de sucesso	Número de bugs encontrados	Qualidade percebida pelo usuário e estabilidade
Papel na produção	Limitado ou inexistente	Ativo: monitora, analisa métricas e feedback

Princípios do QA em DevOps

Abordagem Shift-Left

O **princípio do teste antecipado** às vezes é chamado de shift-left porque é uma abordagem em que o teste é realizado **mais cedo** no SDLC*. Normalmente, o shift-left sugere que os testes devem ser feitos mais cedo (p. ex., não esperar que o código seja implementado ou que os componentes sejam integrados).



*Software Development Life Cycle (SDLC)

Princípios do QA em DevOps

O que envolve o Shift-Left na prática?

- Escrever testes automatizados já durante o desenvolvimento
- Fazer revisões de código com foco em qualidade
- Aplicar teste de unidade, integração e estáticos logo após o commit (via CI)
- QA participando desde o refinamento de requisitos
- Validar histórias de usuário, critérios de aceitação e fluxos críticos desde cedo

Princípios do QA em DevOps

Automação

Em DevOps, **velocidade** e **confiabilidade** são essenciais. A automação de testes permite:

- Detectar erros rapidamente.
- Reduzir esforço manual repetitivo.
- Garantir que funcionalidades não quebrem com mudanças.
- Sustentar a entrega contínua (CI/CD).

Princípios do QA em DevOps

Boas Práticas na Automação:

- Automatize testes estáveis e repetitivos.
- Use versionamento de testes junto ao código-fonte.
- Rode os testes em pipelines após cada commit ou pull request.
- Classifique os testes por tipo e prioridade (ex: smoke, regressão, sanity).
- Monitore tempo de execução: testes muito demorados devem ser otimizados.



Exemplo Prático – Netflix

Desafios

- Alta demanda por novos recursos.
- Escalabilidade limitada pelos métodos tradicionais.
- Complexidade operacional global.

Abordagem DevOps

- Microsserviços + Cloud AWS → escalabilidade e resiliência.
- Automação (CI/CD) → testes e deploys rápidos e seguros.
- Chaos Engineering (Simian Army) → resiliência comprovada.
- Cultura colaborativa e experimental → inovação contínua.

Resultados

- 🚀 Releases mais rápidos e frequentes.
- 💡 Alta disponibilidade e qualidade do software.
- 🌐 Experiência do usuário aprimorada e expansão global.
- 📈 Inovação constante e vantagem competitiva no streaming.



INTERVALO!



20 min.



3. Integração e Ferramentas

Entender como integrar ferramentas
para otimizar o fluxo de qualidade.

Aula 1



Continuous Integration (CI)

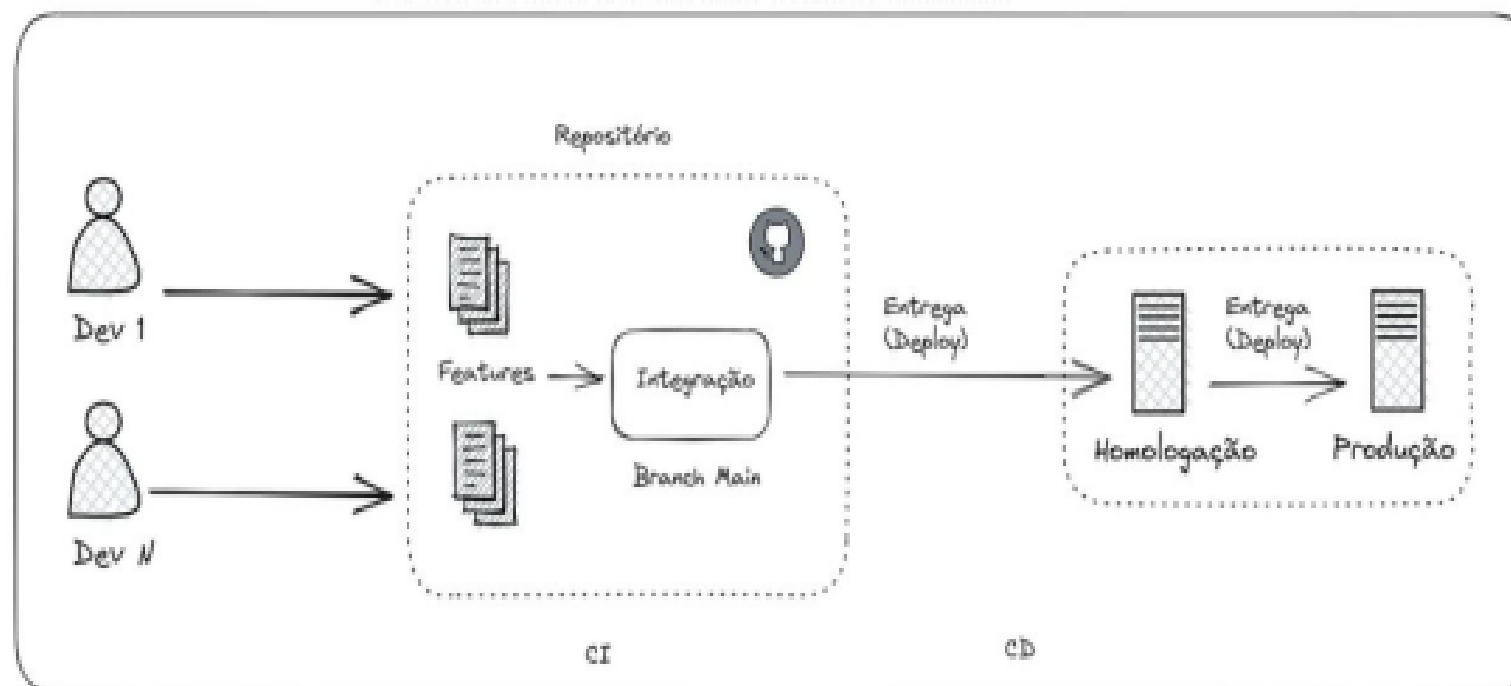
Um **processo/prática** de **automação** que facilita a mesclagem (junção) mais frequente de alterações de código dos desenvolvedores em ramificação compartilhada, ou "tronco".

● Benefícios

- Cada alteração de código é **integrada, compilada e testada automaticamente**.
- Execução de diferentes níveis de testes automatizados (geralmente unitários e de integração).
- Ajuda a encontrar **erros** logo após o commit.
- **Reduz falhas em produção** e mantém o ritmo das **entregas contínuas**.

Continuous Delivery (CD)

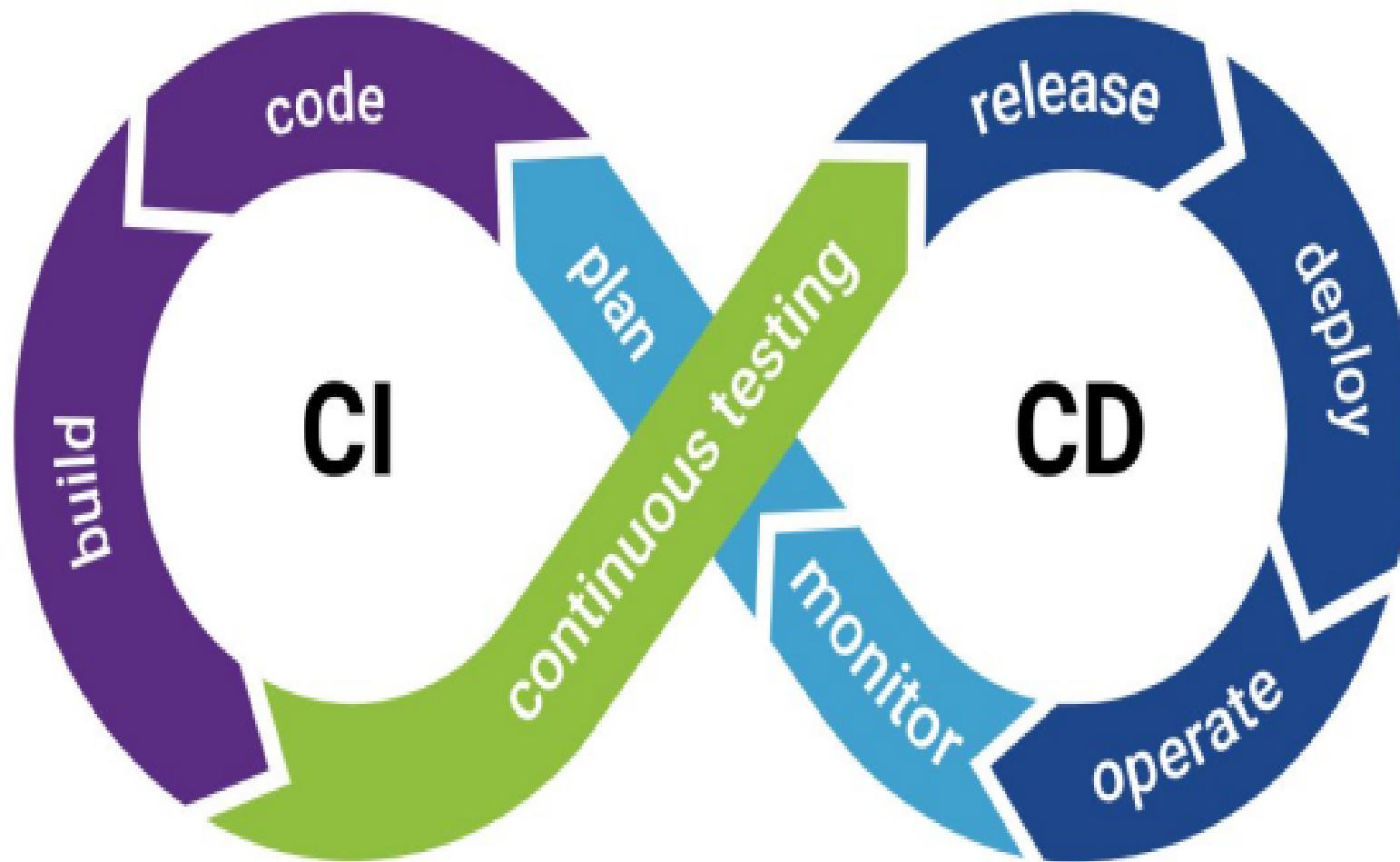
A Entrega Contínua é uma extensão da CI que se concentra em automatizar a **construção**, o **teste** e a **implantação** de uma solução para um **ambiente de produção** ou **homologação**. A entrega contínua garante que o código esteja sempre em um estado pronto para implantação em produção, tornando a implantação um processo simples e repetível.



Fluxo Básico de CI/CD

1. Um desenvolvedor envia código para um repositório Git;
2. Um servidor de integração (CI) monitora o repositório e detecta a alteração;
3. O servidor CI realiza a construção do código e executa testes automatizado;
4. Se os testes forem bem-sucedidos, o código é implantado automaticamente em um ambiente de homologação;
5. Testes adicionais, como testes de aceitação, são executados no ambiente de homologação;
6. Se todos os testes forem aprovados, o código é implantado em produção;
7. O aplicativo é monitorado em produção, e quaisquer problemas são detectados e tratados imediatamente.

CI/CD



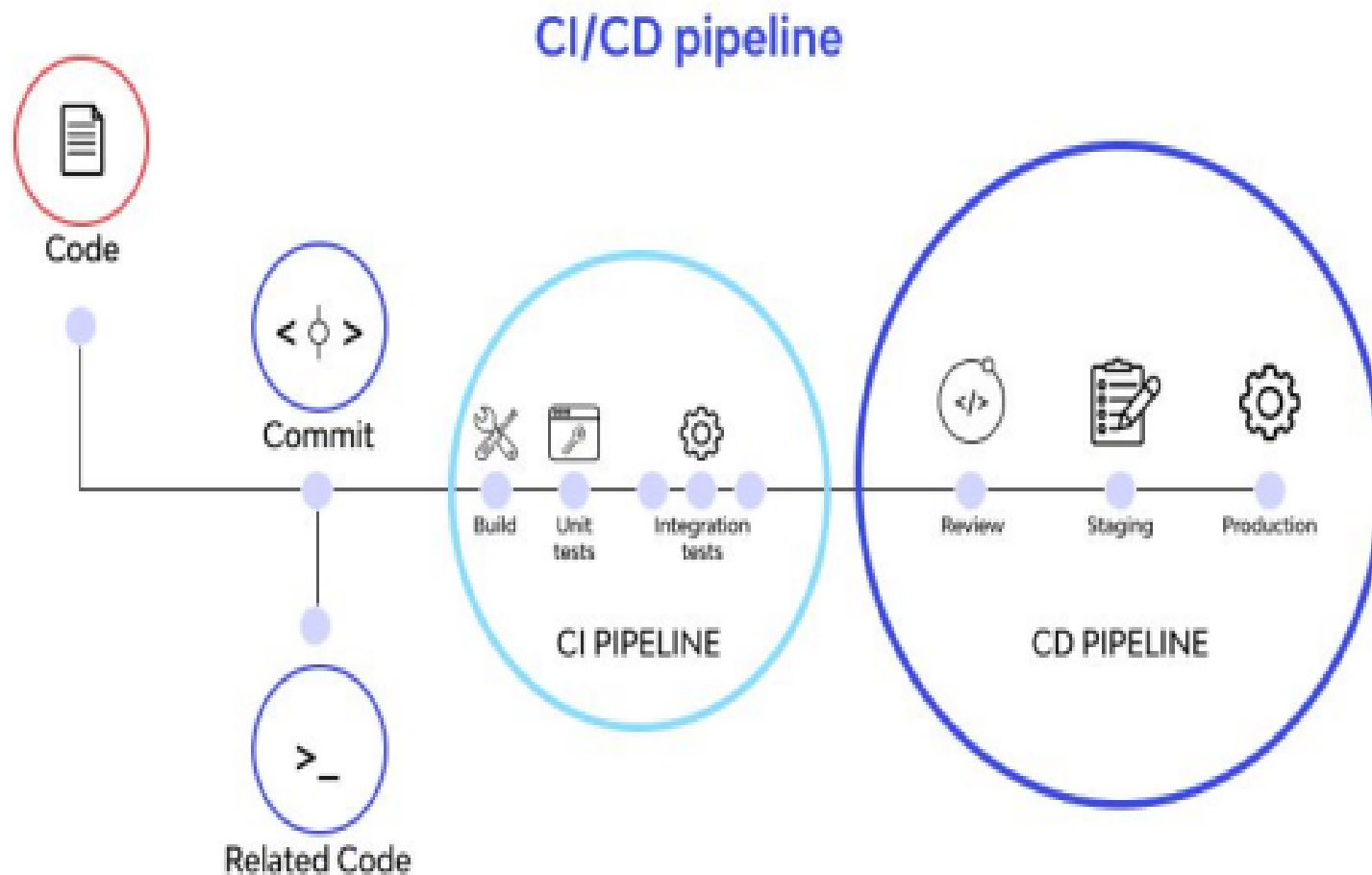
CI/CD Pipeline

Conjunto automatizado de **etapas** ou **processos** que são executados para atingir um objetivo específico. Quase todas as ferramentas utilizadas no pipeline de DevOps devem facilitar a colaboração eficaz entre os diferentes membros da equipe.

💡 Exemplo rápido:

- **Sem** pipeline CI/CD: cada desenvolvedor envia código e alguém precisa testar e instalar manualmente → erros frequentes e processos lentos.
- **Com** pipeline: tudo acontece automaticamente → software confiável e entrega mais rápida.

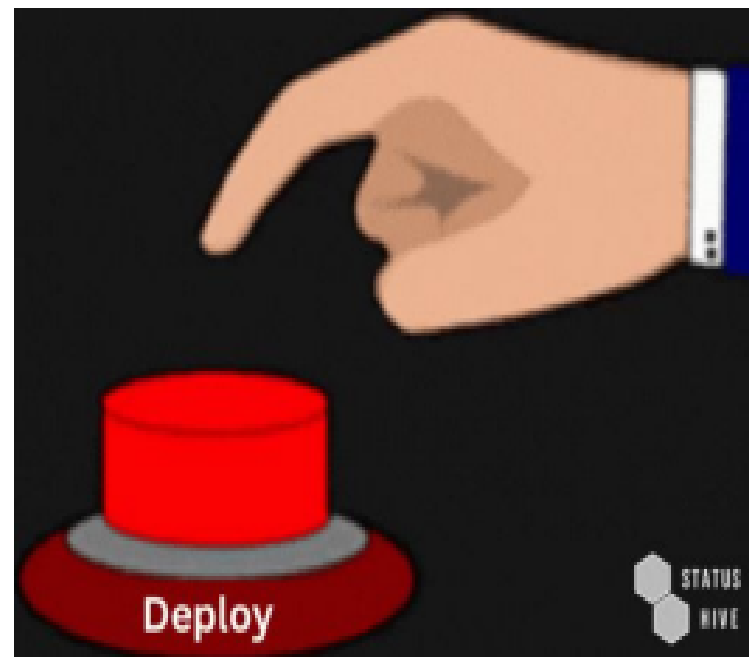
CI/CD Pipeline



Confira as etapas de um Pipeline CI CD. (Fonte: ISmileTechnologies)

Deploy

O processo de Deploy (*do inglês implantar*) significa tornar uma aplicação, software ou site disponível para uso, seja em um **ambiente** de **desenvolvimento**, **teste** ou **produção**. É o processo de colocar o código desenvolvido em um ambiente onde os usuários podem acessá-lo e utilizá-lo. Ferramentas como scripts de implantação e pipelines de CI/CD garantem um deploy consistente e ágil, minimizando o tempo de inatividade da aplicação.



Noções Básicas de Teste Estático

Ao contrário dos testes dinâmicos, nos **testes estáticos** o software em teste **não precisa ser executado**. O teste estático pode ser aplicado tanto para verificação quanto para validação. A análise estática pode identificar problemas antes dos testes dinâmicos e, muitas vezes, exige menos esforço, já que não são necessários casos de teste e, normalmente, são usadas ferramentas.

Exemplos de Ferramentas de análise estática:

- Verificadores ortográficos
- Ferramentas de elegibilidade
- Verificação de boas práticas ou arquitetura de projeto

...



Ferramentas Básicas

Categoria	Ferramentas Populares
CI/CD	Jenkins, GitLab CI, GitHub Actions, Azure DevOps
Automação de Testes	Selenium, Cypress, Playwright, Appium
Testes de API	Postman, Newman, RestAssured, Karate
Gerenciamento de Testes	TestRail, Zephyr, Xray
Observabilidade	Grafana, Prometheus, New Relic
Qualidade de Código	SonarQube

Jenkins


Jenkins

Admin log out

Dashboard > creating-a-pipeline-in-blue-ocean > First-pull-branch >

Status

Changes

Build Now

View Configuration

Full Stage View

Open Blue Ocean

GitHub

Pipeline Syntax

Build History

Filter builds...

 #1

Aug 8, 2022, 2:14 PM

 #2

Aug 8, 2022, 9:48 PM

 #2

Aug 3, 2022, 2:18 PM

Branch First-pull-branch

Full project name: creating-a-pipeline-in-blue-ocean/First-pull-branch

Stage View

Average stage times:
(Average full run time: ~1h 35min)

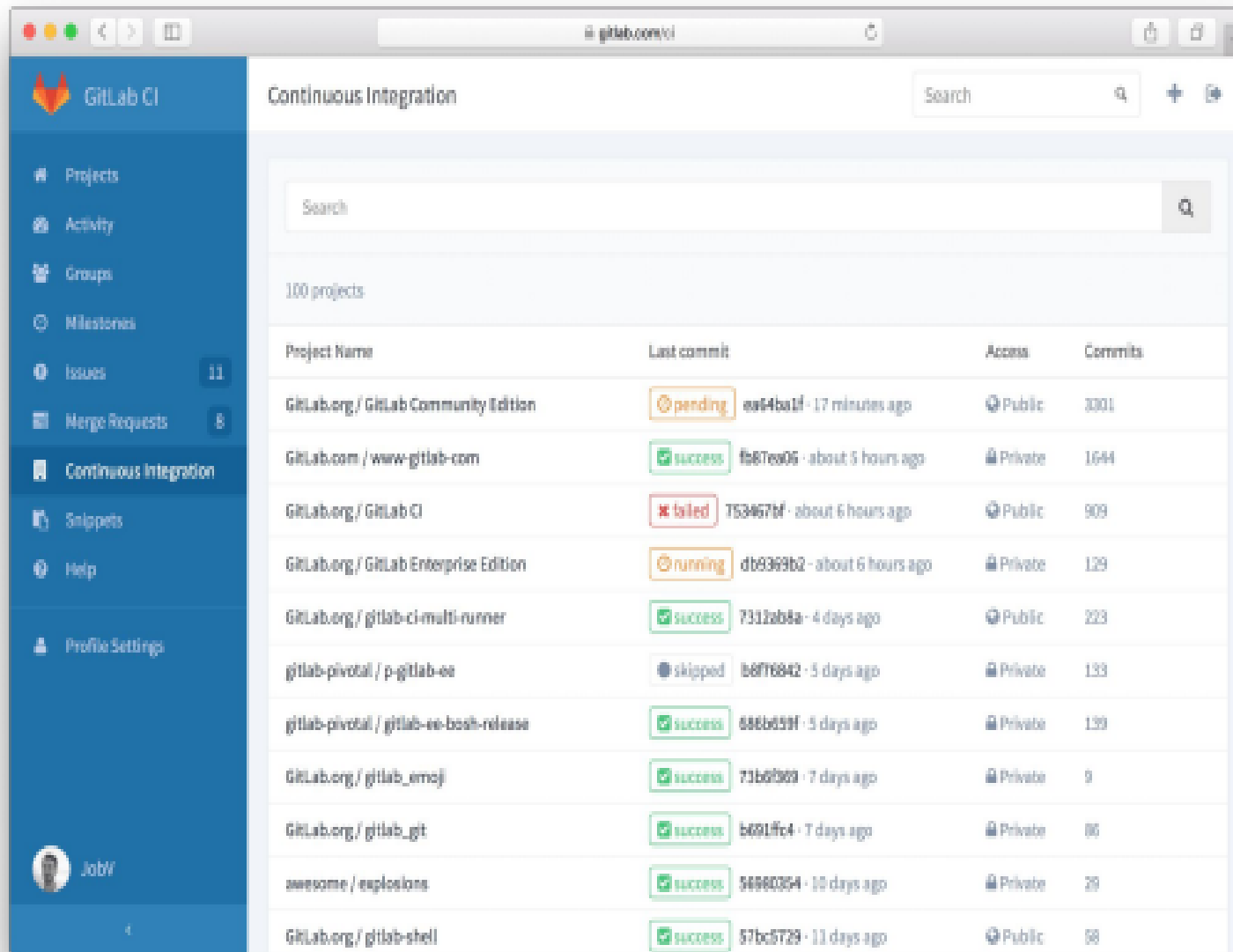
	Declarative Checkout SCM	Build	Test	Test	error	Deliver
#1 Aug 08 14:14 No Changes	1s	34s	753ms	0ms	159ms skipped for 1h 35min	2s skipped for 4min 24s
#2 Aug 04 20:49 No Changes	472ms	21s	37ms	1s	249ms skipped for 34 100% aborted	69ms aborted
#3 Aug 03 14:18 No Changes	7min 27s					

Permalinks



Site Oficial: <https://www.jenkins.io/>

GitLab

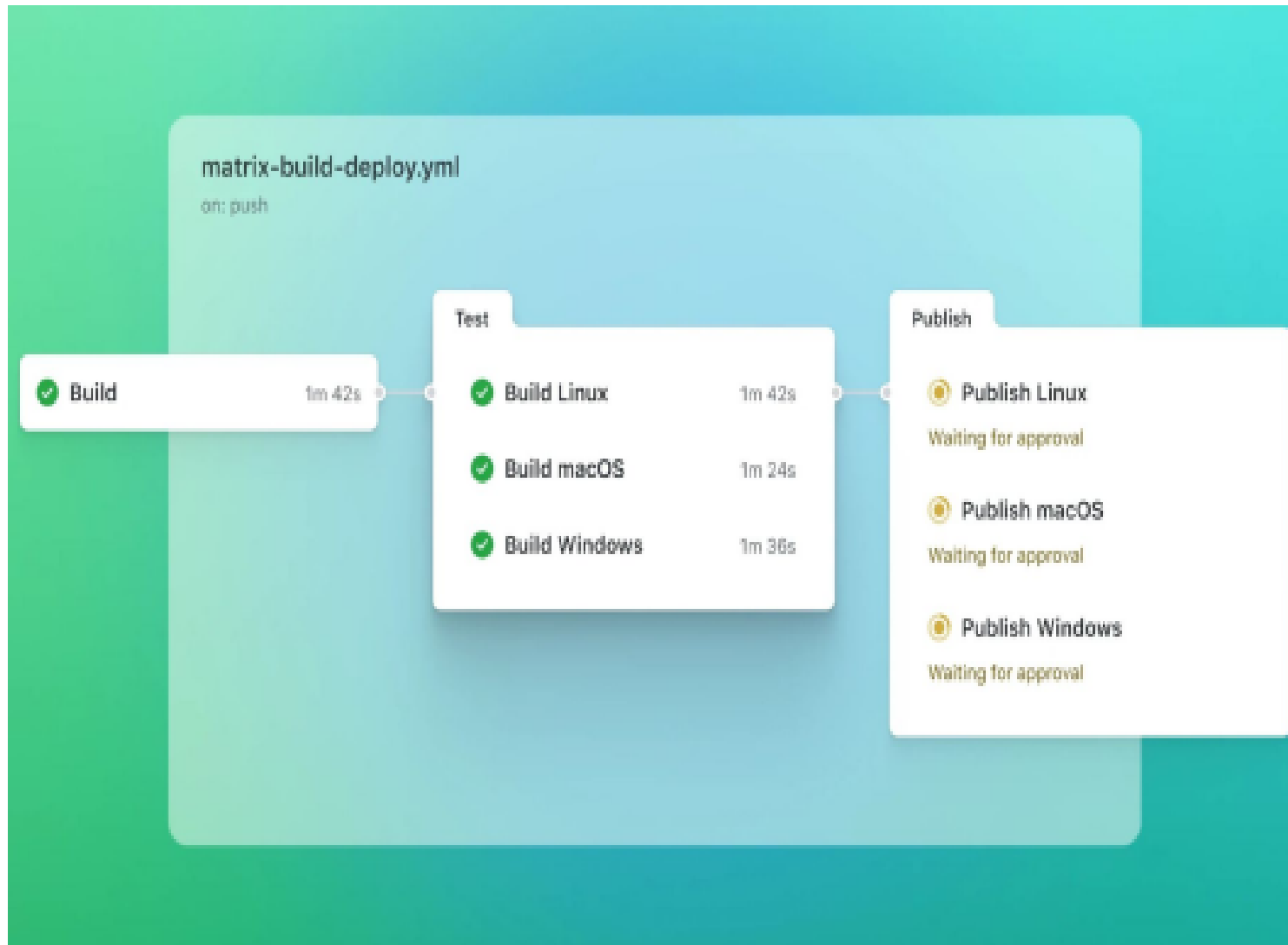


The screenshot displays the GitLab CI web interface. The left sidebar contains navigation links: Projects, Activity, Groups, Milestones, Issues (11), Merge Requests (8), Continuous Integration (selected), Snippets, Help, and Profile Settings. The main content area is titled "Continuous Integration" and features a search bar. Below the search bar, a table lists 100 projects with columns for Project Name, Last commit, Access, and Commits. The table shows various projects with different build statuses: pending, success, failed, running, and skipped.

Project Name	Last commit	Access	Commits
GitLab.org / GitLab Community Edition	pending ea4ba1f · 17 minutes ago	Public	3301
GitLab.com / www-gitlab-com	success fb87ea06 · about 5 hours ago	Private	1644
GitLab.org / GitLab CI	failed 753467bf · about 6 hours ago	Public	909
GitLab.org / GitLab Enterprise Edition	running db9369b2 · about 6 hours ago	Private	129
GitLab.org / gitlab-ci-multi-runner	success 7312ab8a · 4 days ago	Public	223
gitlab-pivotal / p-gitlab-ee	skipped b6ff16842 · 5 days ago	Private	133
gitlab-pivotal / gitlab-ee-bootstrap-release	success 686bd59f · 5 days ago	Private	139
GitLab.org / gitlab_emoji	success 73b6f923 · 7 days ago	Private	9
GitLab.org / gitlab_git	success b691fc4 · 7 days ago	Private	86
awesome / explosions	success 56960354 · 10 days ago	Private	29
GitLab.org / gitlab-shell	success 57bc5729 · 11 days ago	Public	58

Site Oficial: <https://about.gitlab.com/>

GitHub Actions



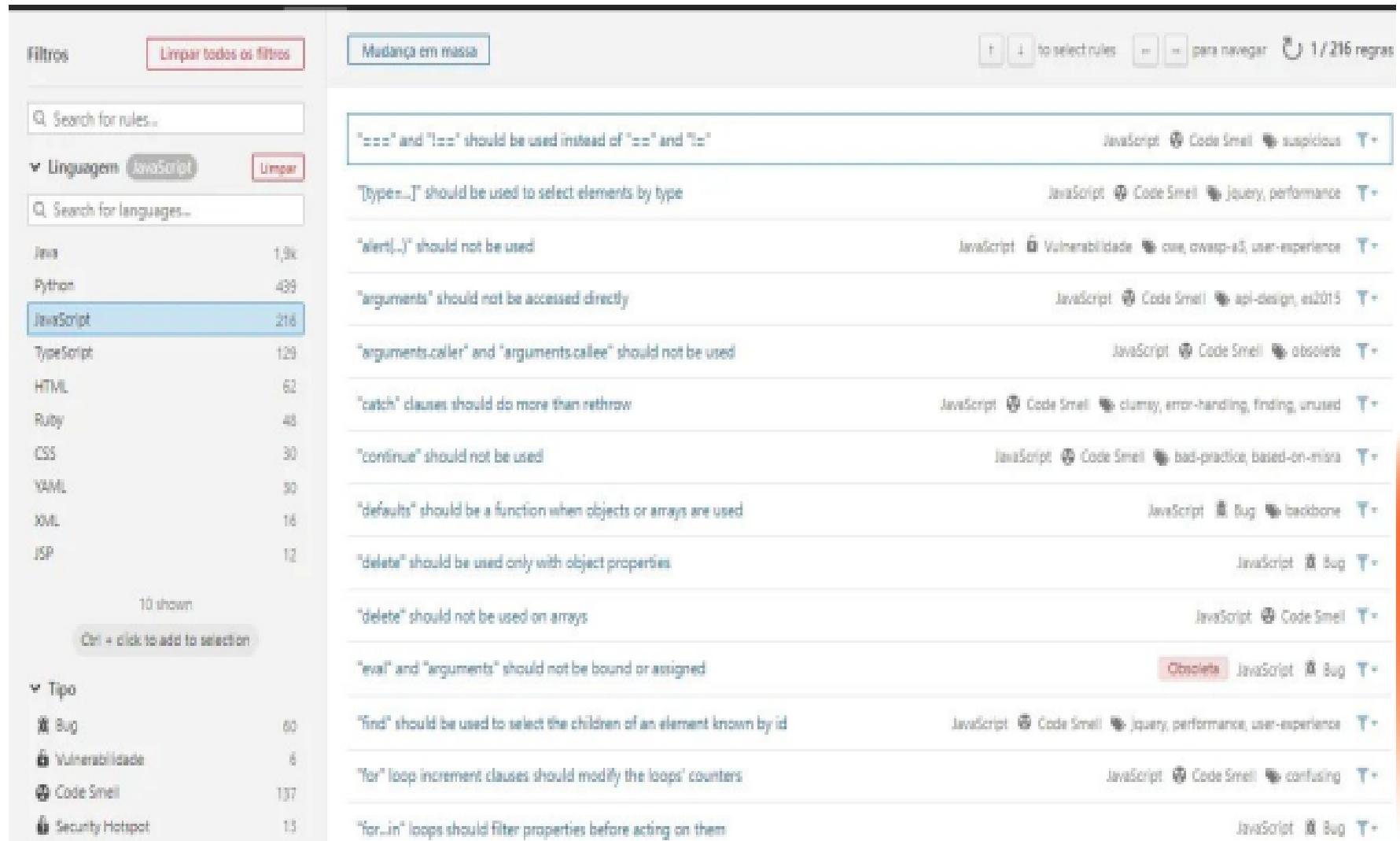
Site Oficial: <https://github.com/features/actions>

Grafana



Site Oficial: <https://grafana.com/>

SonarQube



The screenshot shows the SonarQube interface for managing rules. On the left, there's a sidebar with filters for language and type. The main area displays a list of rules for JavaScript, with details for each rule including its name, category, and tags.

Filtros Limpar todos os filtros

Linguagem JavaScript Limpar

Search for rules...

Search for languages...

Linguagem	Contagem
Java	1,9k
Python	439
JavaScript	216
TypeScript	129
HTML	62
Ruby	48
CSS	30
YAML	30
XML	16
JSP	12

10 shown

Ctrl + click to add to selection

Tipo

Tipo	Contagem
Bug	60
Vulnerabilidade	6
Code Smell	137
Security Hotspot	13

Mudança em massa to select rules para navegar 1 / 216 regras

Regra	Linguagem	Categoria	Tags
"===" and "!== " should be used instead of "==" and "!="	JavaScript	Code Smell	suspicious
"Type<...>" should be used to select elements by type	JavaScript	Code Smell	jquery, performance
"alert(...)" should not be used	JavaScript	Vulnerabilidade	owasp, owasp-a3, user-experience
"arguments" should not be accessed directly	JavaScript	Code Smell	api-design, es2015
"arguments.caller" and "arguments.callee" should not be used	JavaScript	Code Smell	obsolete
"catch" clauses should do more than rethrow	JavaScript	Code Smell	clumsy, error-handling, finding, unused
"continue" should not be used	JavaScript	Code Smell	bad-practice, based-on-misra
"defaults" should be a function when objects or arrays are used	JavaScript	Bug	backbone
"delete" should be used only with object properties	JavaScript	Bug	
"delete" should not be used on arrays	JavaScript	Code Smell	
"eval" and "arguments" should not be bound or assigned	Obsolete	JavaScript	Bug
"find" should be used to select the children of an element known by id	JavaScript	Code Smell	jquery, performance, user-experience
"for" loop increment clauses should modify the loops' counters	JavaScript	Code Smell	confusing
"for...in" loops should filter properties before acting on them	JavaScript	Bug	

Site Oficial: <https://www.sonarsource.com/>

ESLint

```
1 // Imports
2 import mongoose, { Schema } from 'untitled'
3
4 // Collection name
5 export const collection = 'Design'
6
7 // Schema
8 const schema = new Schema({
9   name: {
10     type: String,
11     required: true
12   },
13
14   description: {
15     type: String
16   }
17 }, {timestamps: true})
18
19 // Model
20 export default untitled.model(collection, schema,
21 collection)
```

```
vagrant@precise32:~$ eslint uploader.js
```

```
uploader.js
```

```
1:28 error Strings must use doublequote
7:15 error Strings must use doublequote
7:28 error Strings must use doublequote
7:34 error Missing "use strict" statement
9:20 error Expected '===' and instead saw '=='
10:17 error Expected '===' and instead saw '=='
14:20 error Strings must use doublequote
35:1 error Trailing spaces not allowed
36:12 error Strings must use doublequote
45:1 error Unexpected blank line at end of file
```

Rule names

quotes
quotes
quotes
strict
eqeqeq
eqeqeq
quotes
no-trailing-spaces
quotes
eol-last

```
& 10 problems
```

```
vagrant@precise32:~$
```

ESLint

```
js  
  
var nome = "Natasha"  
console.log("Oi")
```

Por que testes dinâmicos não identificariam esse erro diretamente?

ESLint

Um teste dinâmico (ou seja, rodando a aplicação com testes de execução: unitários, de integração, etc.) não pegaria esse erro diretamente, porque:

- O código roda normalmente, sem quebrar.
- Não há falha em tempo de execução: o `console.log("Oi")` funciona.
- A única “falha” é de qualidade/manutenção (variável inútil ocupando espaço).
- ESLint apontaria que a variável nome nunca foi usada.



4. Qualidade e Feedback Contínuo

Identificar práticas que garantem qualidade ao longo do ciclo de desenvolvimento.

Aula 1



Práticas para Garantir Qualidade

Testes Automatizados em Diferentes Níveis

- Unitários, integração, end-to-end, testes de API.
- Exemplo: Pipeline roda testes unitários a cada commit.

Code Review Contínuo

- Pull Requests + análise automatizada (SonarQube, ESLint).

Integração Contínua (CI)

- Cada commit aciona build e testes automáticos.
- Reduz riscos de bugs acumulados.

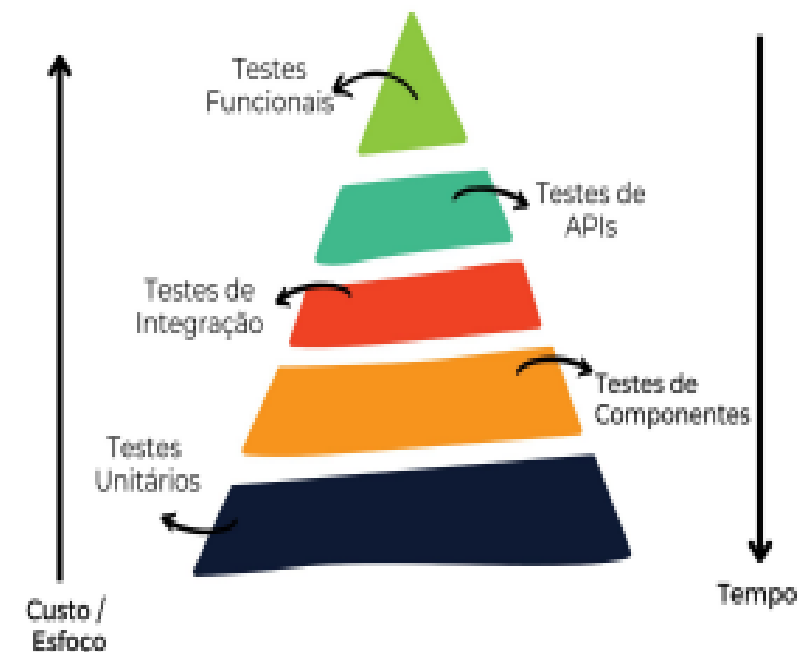
Monitoramento Contínuo

- Observabilidade (logs, métricas, alertas).
- Detecta falhas em produção rapidamente.

Testes de Performance e Segurança Automatizados

- Incluídos no pipeline, não só no final.

Pirâmide de Testes



Feedback Contínuo

Informação recebida rapidamente sobre algo que afeta a qualidade:

- Permite corrigir rápido antes do impacto no usuário.
- Dá visibilidade para todos: Dev, QA, Ops, Negócio.
- Conexão com Controle de Qualidade:
 - Não é só detectar defeitos → é prevenir, ajustar, melhorar.

 **All checks have failed**
1 failing check

  Check / check (pull_request) Failing after 1m — check

 **Merging is blocked**
Merging can be performed automatically with 1 approving review.

DÚVIDAS?!
:)





C . E . S . A . R

MUITO OBRIGADA!

© CESAR | Todos os direitos reservados